TECHNICAL PROJECT REPORT

Building a Modern Transformer Language Model from Scratch

Implementing and Benchmarking Grouped Query Attention & KV Cache

Author: Geetesh Jangir
Repository: github.com/Geetesh-Jangir/custom-llm-with-GQA
Stack: PyTorch · Python · from-scratch decoder-only Transformer

Project Snapshot

- 7.43× faster generation with KV Cache — 141.5 → 1,051.8 tokens/s
- 8.83% lower peak GPU memory using Grouped Query Attention vs. standard MHA
- Stable generation up to 2,048-token context with peak memory under 0.18 GB
- Built entirely from scratch in PyTorch: GQA, RoPE, RMSNorm, SwiGLU, KV Cache, Sliding Window Attention, and an SFT pipeline

1 Project Overview

Most educational Transformer implementations stop at the textbook architecture and skip the engineering that makes large language models usable in production. This project closes that gap: a decoder-only, GPT-style language model was built entirely from scratch in PyTorch and extended with the optimizations that modern open-source LLMs rely on — Grouped Query Attention (GQA), a Key-Value (KV) Cache, Rotary Positional Embeddings (RoPE), RMSNorm, a SwiGLU feed-forward network, and Sliding Window Attention.

Rather than treating these techniques as black boxes inside pre-trained frameworks, each one was implemented directly and benchmarked under controlled conditions to measure its real contribution to inference latency, throughput, and memory footprint.

2 Objectives

- Build a decoder-only, GPT-style language model from scratch.
- Implement the modern Transformer optimizations used in contemporary open-source LLMs.
- Evaluate the effect of Grouped Query Attention (GQA) against standard Multi-Head Attention.
- Evaluate the effect of a Key-Value (KV) Cache during autoregressive inference.
- Analyze memory usage and behavior under long-context generation.

- Gain practical, low-level experience in LLM engineering beyond what pre-trained frameworks expose.

3 System Architecture

The model follows a decoder-only Transformer architecture in the style of GPT. Each of the N stacked blocks applies RMSNorm before Grouped Query Attention (with RoPE-encoded queries and keys, and KV-Cache reuse during generation), and again before a SwiGLU feed-forward sub-layer — each wrapped in a residual connection.

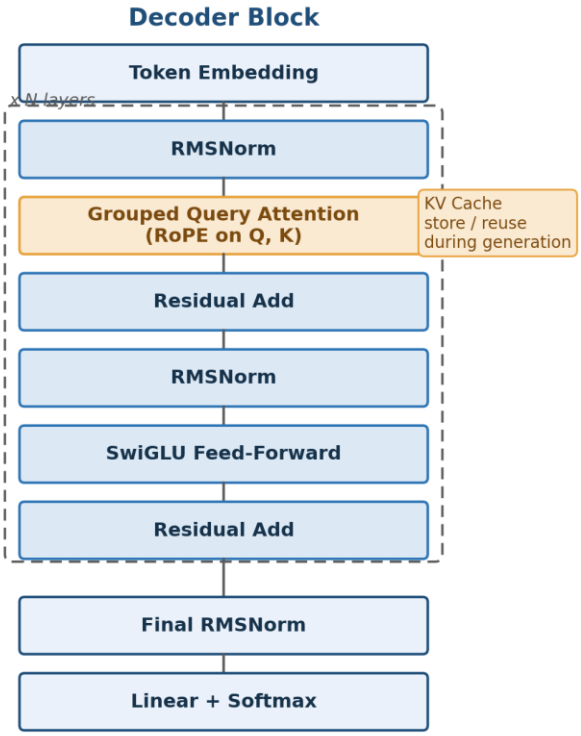

Figure 1. Decoder block used in this work, showing the repeated RMSNorm → GQA → residual → RMSNorm → SwiGLU → residual stack.

4 Core Components

4.1 Grouped Query Attention (GQA)

Instead of giving every attention head its own Key and Value projection, GQA groups multiple Query heads to share a smaller number of Key-Value heads. This shrinks the Key/Value projections and the resulting KV Cache while largely preserving representational capacity relative to full Multi-Head Attention (MHA).

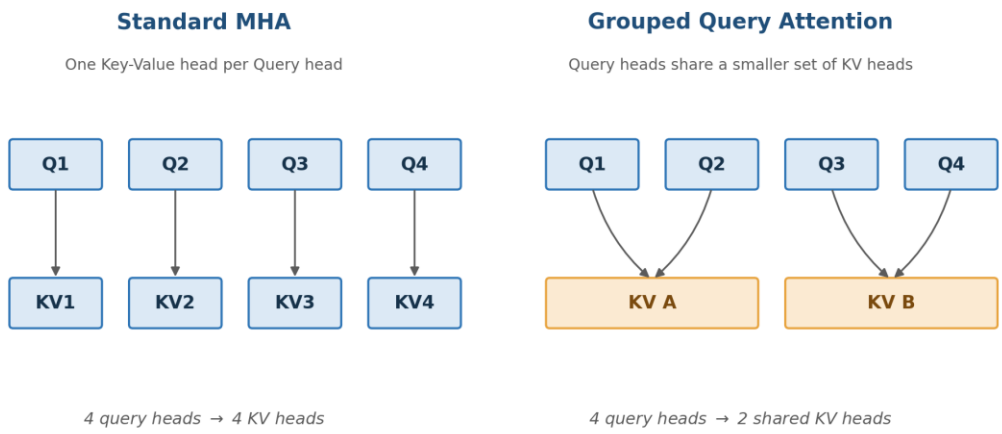

Figure 2. Standard MHA assigns one Key-Value head per Query head (left); GQA shares each Key-Value head across a group of Query heads (right), shrinking the projections and the KV Cache.

4.2 KV Cache

Autoregressive generation would otherwise re-encode every previously generated token at each step. The KV Cache stores previously computed Key and Value tensors so only the newest token's representations need to be computed at each step, avoiding quadratic recomputation as the sequence grows.

4.3 Rotary Positional Embeddings (RoPE)

RoPE replaces learned absolute positional embeddings, encoding relative position directly into the Query and Key vectors via rotation — an approach adopted by most modern open-source LLMs for its extrapolation properties.

4.4 RMSNorm

RMSNorm replaces LayerNorm throughout the network, removing the mean-centering step to reduce computational overhead while preserving training stability.

4.5 SwiGLU Feed-Forward Network

The feed-forward sub-layers use the SwiGLU activation, widely adopted in modern architectures for its improved parameter efficiency and representational power relative to standard ReLU/GELU MLPs.

4.6 Additional Components

- Sliding Window Attention and attention sink tokens
- Mixed-precision training and gradient accumulation
- Byte-level and BPE tokenization
- A Supervised Fine-Tuning (SFT) pipeline

5 Experimental Setup

All experiments use the same underlying model implementation for a fair, controlled comparison. The evaluation targets three questions: (1) how much does KV Cache improve inference speed; (2) what benefit does GQA provide over standard MHA; and (3) how does the model behave as context length grows. For each setting, generation time, throughput (tokens/s), peak GPU memory, and relative speedup were recorded.

6 Results & Benchmarks

6.1 KV Cache Evaluation

Table 1 compares autoregressive generation with and without KV Cache.

Metric	No KV Cache	With KV Cache
Generation time (s)	3.534	0.475
Throughput (tok/s)	141.48	1051.77
Speedup	1.00×	7.43×

Throughput increases from 141.48 to 1051.77 tokens per second — a measured 7.43× speedup — confirming KV Cache as a critical inference-time optimization. Without it, generation cost grows substantially as the sequence lengthens.

6.2 Grouped Query Attention Evaluation

Table 2 compares standard MHA against GQA, which reduces the number of Key-Value heads while preserving the number of Query heads.

Metric	MHA	GQA
Generation time (s)	3.899	3.697
Peak GPU memory (GB)	0.263	0.239
Throughput (tok/s)	128.23	135.24
Memory reduction	–	8.83%

GQA yields measurable memory savings alongside a slight throughput improvement. The absolute reduction is modest at this model scale, but it compounds across layers, heads, and longer sequences — which is why GQA is widely adopted in large production models.

6.3 Long-Context Evaluation

Table 3 reports generation behavior as context length grows from 128 to 2,048 tokens.

Context	Time (s)	Tok/s	Mem (GB)
128	2.703	73.98	0.11

Context	Time (s)	Tok/s	Mem (GB)
256	0.718	278.68	0.11
512	2.364	84.59	0.12
1024	0.497	402.05	0.14
2048	1.603	124.78	0.18

The model sustains generation at context lengths up to 2,048 tokens while keeping memory consumption moderate — attributable to the combined effect of GQA, KV Cache, and Sliding Window Attention.

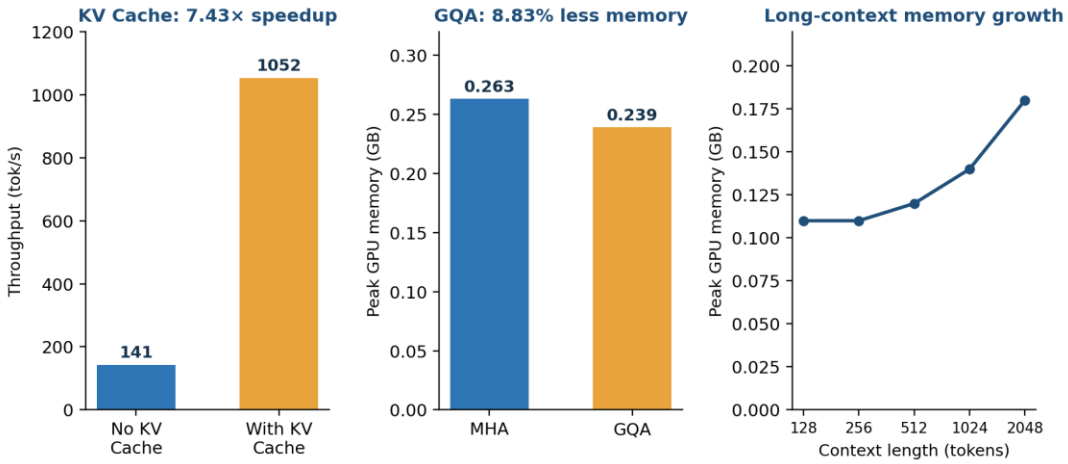

Figure 3. KV Cache delivers the largest throughput gain (left); GQA trims peak memory at equal head count (center); peak memory grows gradually with context length (right).

7 Key Learnings

KV Cache is essential

The largest improvement observed in this work comes from KV Cache. The measured 7.43× speedup explains why virtually all production LLM inference stacks rely on cached attention states.

GQA improves efficiency

GQA reduces memory usage and improves throughput while keeping the architecture simple, clarifying how modern models achieve better scalability without materially increasing computational cost.

Engineering matters as much as architecture

A textbook Transformer is often insufficient for practical deployment. Optimizations such as KV Cache, RMSNorm, SwiGLU, and efficient attention mechanisms substantially affect real-world performance.

Benchmarking is critical

Implementing a feature is only half the work; controlled measurement is what demonstrates that an optimization is actually beneficial.

8 Future Work

- Flash Attention
- LoRA fine-tuning
- INT8 / 4-bit quantization
- Distributed and multi-GPU training
- Retrieval-Augmented Generation (RAG)
- Reinforcement Learning from Human Feedback (RLHF)
- Evaluation at larger context windows

9 Conclusion

This project delivers a complete, from-scratch implementation and controlled evaluation of a modern decoder-only Transformer language model in PyTorch. KV Cache dramatically improves inference performance, achieving a measured 7.43× speedup, while Grouped Query Attention reduces memory requirements by 8.83% and modestly improves throughput. The architecture additionally sustains generation at context lengths up to 2,048 tokens with peak memory remaining under 0.18 GB.

Beyond the numbers, the main outcome of this project is a clear, first-hand understanding of how the architectural and systems-level optimizations underlying modern LLMs interact to enable efficient, scalable deployment.

10 Project Repository

Full source code, model implementation, and benchmarking scripts are available on GitHub:

https://github.com/Geetesh-Jangir/custom-llm-with-GQA